Is There Anything to Search For? A Counterfactual Head-Room Analysis of Test-Time Action Selection for VLA Models

Anonymous Author(s)
Affiliation
Address
email

Abstract: Test-time selection—sampling candidate actions from a robot policy and executing the one a learned scorer prefers—is a popular way to make vision-language-action (VLA) models more robust without retraining them. On RynnVLA-002, whose success on LIBERO-Goal drops from 98.0% to 52.6% when the robot's initial state is perturbed, we built a latent imagination search over 16 proposed action chunks and evaluated five scorers on the same 100 paired episodes: none beat the unmodified policy, and the two that overrode it most often were significantly worse (-13 and -28 points). Diagnostics show that the scorers reject implausible chunks but cannot rank plausible ones, and that outcome-trained scorers predict *whether* an episode will succeed (AUC up to 0.915) rather than *which* chunk to take. To ask whether any scorer could have helped, we restore simulator snapshots and replay alternative chunks from the same states: replacing a single chunk with any of three alternatives, after which the policy resumes control, rescues only 4.9% of failure states, bounding the gain of a perfect single-chunk selector at 7.0 points. Under these perturbations, failures are rarely one alternative chunk away from success; whether interventions over several consecutive decisions can close this action gap is left open.

Keywords: Vision-language-action models, test-time action selection, world models

1 Introduction

VLA models such as RT-2 [1], OpenVLA [2] and π_0 [3] achieve near-saturated success on standard simulation benchmarks, but robustness studies show steep drops under modest changes to the scene or the robot. LIBERO-Plus [4], for example, perturbs LIBERO [5] along several axes; we find that the RynnVLA-002 [6] Goal policy falls from 98.0% to 52.6% when only the robot's initial state is changed (Table 1).

A natural, training-free remedy is *test-time selection*: sample several candidate actions and let a verifier, value function or world model choose. RoboMonkey [7] and V-GPS [8] report gains from such re-ranking, and unified models that already contain a world model [9, 6] suggest imagining each candidate's consequences before acting, as in latent-space planning [10, 11, 12].

We set out to build exactly this and failed to make it work. Instead of only reporting the failure, we use it to ask a more basic question: *in the states where the policy fails, does a better action chunk exist among those a proposer can produce?* If not, no scorer—however well trained—can help. Our contributions are:

1. A controlled, paired evaluation of five test-time selection variants on the same 100 perturbed episodes, none of which beats the base policy (Section 4).

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

2. Diagnostics separating two failure modes of the scorers: an optimiser's-curse effect [13] when overriding the policy on noisy scores, and outcome-trained scorers that learn the value of the *state* rather than of the *action* (Section 5).
3. A counterfactual branching protocol that measures the head-room of chunk-level selection directly by restoring simulator snapshots, and the finding that the head-room of a single-chunk intervention is small: an oracle upper bound only 7.0 points above the policy (Section 6). We make no claim about interventions that span several decision points.

Code, scripts and all raw results are available at the supplementary material (released upon publication).

2 Related work

Test-time selection for robot policies. RoboMonkey [7] samples and perturbs actions from a VLA and selects one with a learned VLM-based verifier; V-GPS [8] re-ranks samples of a generalist policy with a value function trained by offline RL. Both report gains; we study the same recipe under robot initial-state perturbations, and add a direct measurement of how much any selector could gain.

World models for action selection. Latent-space planning with learned dynamics, from PlaNet [10] and DreamerV3 [14] to TD-MPC2 [11] and DINO-WM [12], chooses actions by imagining their consequences. Unified VLAs such as WorldVLA [9] and RynnVLA-002 [6] contain an action-conditioned world model, but in RynnVLA-002's released evaluation code actions come from the action head alone. Our search follows this idea in a small latent space attached to RynnVLA-002, with goals given by language rather than images or rewards.

Why imitation policies fail. Covariate shift—a policy trained on expert states drifting into states it cannot recover from—is the classic explanation for compounding errors in imitation learning [15]. Our branching results are consistent with this account for VLAs under initial-state perturbations, and suggest that it limits test-time selection as well.

3 Setup

Policy (System 1). RynnVLA-002 [6] is a 7B Chameleon-style [16] autoregressive model trained both as a policy and as an action-conditioned image predictor. We use its released LIBERO-Goal checkpoint (256 px, two history frames, third-person and wrist views, proprioceptive state, chunks of $H{=}5$ actions predicted by its continuous action head) unchanged, with its evaluation settings.

Benchmark. We evaluate on LIBERO-Plus [4] Goal. Selection experiments use a fixed set of 100 episodes from the "Robot Initial States" category (the first 200 variants of the category, shard 0 of 2); System 1 solves 50 of them. Every variant is compared with System 1 on exactly these episodes, with a paired bootstrap interval and an exact McNemar test. All data used to train scorers come from LIBERO *training* tasks with our own random joint perturbations (Gaussian noise with a per-episode standard deviation of up to 0.3 rad added to the arm joints); no LIBERO-Plus variant is ever used for training or tuning. The same 100 test episodes were, however, reused as we moved from one selection variant to the next; since no variant improved on System 1, this reuse cannot have inflated a positive result, but it means the numbers in Table 2 are not from a single pre-registered comparison.

Search (System 2). Our selection module, LIS (latent imagination search), adds about 50M trainable parameters on top of the frozen backbone and reuses its single forward pass. A flow-matching [17, 18] proposer samples 15 chunks, to which System 1's chunk is added as candidate 1; a latent world model predicts, for each chunk, a pooled latent of the observation H steps later; a scorer assigns each candidate a value and the best is executed. The proposer, world model and a language-similarity evaluator are trained on LIBERO-Goal demonstrations only, with a JEPA-style

Table 1: System 1 (RynnVLA-002, released checkpoints) success rates. 95% Wilson intervals.

Benchmark	Episodes	Success (%)	95% CI
LIBERO-Goal (standard)	100	98.0	[93.0, 99.4]
LIBERO-Plus Goal	834	56.8	[53.4, 60.2]
Objects Layout	425	60.9	[56.2, 65.5]
Robot Initial States	409	52.6	[47.7, 57.4]
LIBERO-10 (standard)	100	94.0	[87.5, 97.2]
LIBERO-Plus 10	705	58.9	[55.2, 62.4]
Objects Layout	312	71.8	[66.6, 76.5]
Robot Initial States	393	48.6	[43.7, 53.5]

Table 2: Test-time selection among 16 candidate chunks versus System 1 alone, on the same 100 LIBERO-Plus Goal episodes with perturbed robot initial states (System 1: 50/100). Override: share of control steps at which a non-System-1 chunk was executed. Wins: episodes solved only by the variant / only by System 1. CI: paired bootstrap; p: exact McNemar test.

Selection rule	Override (%)	Success	Δ (pts)	95% CI	Wins	p
System 1 only	0.0	50	–	–	–	–
Language-similarity scorer, argmax over 16	71.8	37	-13	[-24, -1]	11/24	0.04
Language scorer + z-score gate (p90)	7.9	44	-6	[-13, +1]	3/9	0.15
Language scorer + gate (p75) + S1 prior	0.2	50	+0	[+0, +0]	0/0	1.00
Outcome-trained value of imagined latent, margin 0.05	3.4	49	-1	[-5, +3]	2/3	1.00
Outcome-trained Q(z, ctx, g, A), margin 0.05	44.1	22	-28	[-39, -17]	5/33	4.3e-06

latent prediction loss [19], contrastive language grounding and ranking against perturbed expert chunks (details in the repository; training took 5,000 steps with the backbone frozen).

4 Test-time selection does not help

We evaluate five selection rules (Table 2):

- **Language similarity, argmax.** Score $= \cos(\phi(\hat{\boldsymbol{z}}), \psi(\boldsymbol{g}))$ between the imagined latent and the instruction embedding, minus a smoothness cost; execute the argmax.
- **Gated variants.** Keep System 1's chunk unless the best candidate's score exceeds System 1's by more than a margin measured in score standard deviations (z). Margins are set to the 90th (2.58) and 75th (1.83) percentile of this gap on held-out demonstrations; the second variant also adds a bonus for proximity to System 1 and samples closer to it.
- **Outcome-trained scorers.** We collected 561 System-1 rollouts on the training tasks under our perturbations, executing a random proposer candidate instead of System 1 at 30% of queries (6,613 of 22,419 queries), and trained two scorers to predict discounted episode success: one from the imagined latent ("imag"), one directly from the current latent, context and candidate chunk ("direct"). At test time System 1 is overridden only if a candidate's predicted success probability exceeds System 1's by 0.05.

No variant improves on System 1. Variants that rarely override it are statistically indistinguishable from it, and the two that override it most often are significantly worse (Figure 1). In other words, the more the search is trusted, the more it hurts.

5 Why the scorers fail

The language-similarity scorer has signal, but not where it is needed. On 300 held-out demonstration states, the trained scorer prefers the expert chunk to a noisy copy of it 91.4% of the time at the smallest noise level and 99.8% at the largest. Among *proposer* candidates, however, its scores are uncorrelated with closeness to the expert (Spearman $\rho = 0.06$). Its argmax was never closer to

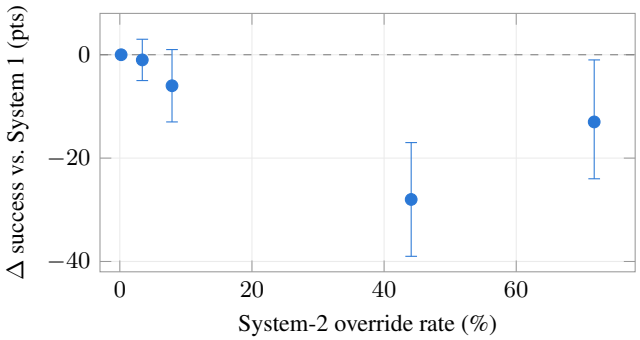


Figure 1: Change in success against how often System 2 overrides System 1, for the five variants in Table 2 (bars: paired-bootstrap 95% CI).

the expert chunk than System 1's chunk (0.0% of states), was farther in 76.3%, and had 4.5× System 1's squared error. The scorer learned to reject implausible, jerky chunks—which the proposer rarely produces—not to rank plausible ones. Taking the maximum of 16 noisy scores then almost always selects a candidate whose advantage is noise: the optimiser's curse [13]. Gating removes most overrides, and with them most of the damage, but not the absence of useful ones.

Outcome-trained scorers learn state value, not action value. Both outcome-trained scorers predict held-out episode success well (AUC 0.797 for "imag" and 0.915 for "direct", on 84 validation episodes). Yet on the validation queries where a non-System-1 chunk was executed, the sign of the scorer's preference between that chunk and System 1's agrees with the episode outcome only 48.6% and 37.6% of the time—at or below chance—and System 1's chunk sits in the middle of the ranking (on average 7.8 and 9.7 of the 15 alternatives are scored above it). Success is largely determined by the state the robot is already in, so the easiest way to predict it is to ignore the candidate. Exploration data with one random alternative per query provide little signal about which chunk is better from the same state.

These findings point to a deeper question. A better scorer could fix both problems—but only if, in the states where System 1 fails, the proposer actually produces chunks that would lead to success.

6 Measuring head-room by counterfactual branching

Protocol. We run System 1 on LIBERO-Goal training tasks with our joint perturbations (a separate set of episodes from those used to train the scorers) and save the full simulator state at every fourth query, together with the observation history and the 16 candidates. After an episode ends, we pick up to three snapshots spread over a failed episode (one in a successful episode), excluding the last. At each such branch point we restore the snapshot and play four branches to the end of the episode: System 1's own chunk, and three chunks drawn at random from the 15 proposer candidates, each followed by System 1. A branch point is *rescuable* if an alternative succeeds while the System-1 branch fails. The fraction of failed episodes with at least one rescuable point gives an oracle upper bound on what a perfect scorer could achieve with this proposer and this budget.

Results. Table 3 summarises the outcome. Replaying System 1's own chunk reproduces the original outcome in 91.9% of branches, so the simulator and policy are close to deterministic and differences between branches are attributable to the chunk. From failure states, a single random alternative succeeds only 2.7% of the time—while from success states it still succeeds 93.1% of the time, so the proposer's chunks are not simply bad. Only 4.9% of failure branch points (8/163, 95% CI up to 9.4%) and 13.3% of failed episodes are rescuable, and the oracle episode success is 54.8% against System 1's 47.8%: a gain of 7.0 points, far below the losses incurred by imperfect scorers in Table 2. Rescue is somewhat more likely at the earliest branch point (10.6%, 5/47) than in the middle (3.6%)

Table 3: Counterfactual branching on 115 LIBERO-Goal training-task episodes with our own robot initial-state perturbations (disjoint from the LIBERO-Plus test variants). At each branch point the simulator is restored and one alternative chunk from the proposer is executed, after which System 1 continues to the end of the episode. 95% Wilson intervals.

Quantity	Count	%	95% CI
System 1 episode success	55/115	47.8	[38.9, 56.9]
System-1 branch reproduces the original outcome	216/235	91.9	[87.7, 94.8]
Failure branch points rescued by $\geq$1 of 3 alternatives	8/163	4.9	[2.5, 9.4]
early branch point	5/47	10.6	[4.6, 22.6]
middle branch point	2/56	3.6	[1.0, 12.1]
late branch point	1/60	1.7	[0.3, 8.9]
Failed episodes rescued at any branch point	8/60	13.3	[6.9, 24.2]
Oracle episode success (System 1 + rescued)	63/115	54.8	[45.7, 63.6]
Single alternative succeeds, from a failure state		2.7	
Single alternative succeeds, from a success state		93.1	

or late (1.7%), consistent with deviations becoming harder to undo as an episode progresses, but it is rare everywhere. Because replaying System 1's own chunk changes the outcome in about one branch in twelve, some of these rescues may reflect simulator or inference nondeterminism rather than a better chunk, so the oracle figure is, if anything, optimistic.

Interpretation. When the perturbed robot starts in an unfamiliar configuration and System 1 fails, the failure is rarely one wrong chunk away from success: after almost any single deviation, System 1 resumes control and fails again. For a selector that changes one decision and then defers to the policy, the bottleneck is therefore the policy's behaviour in these states, not the choice among its candidates. This does not establish the same ceiling for repeated selection: recovery may need several coordinated chunks, and System 1 may undo an initially helpful intervention, both of which our protocol cannot capture. This is consistent with the classic covariate-shift account of imitation learning [15]: a policy trained on expert states cannot recover from states it has not seen, and choosing among its own proposals does not change which states it can recover from.

7 Limitations and discussion

- **Scope.** One policy (RynnVLA-002), one suite (Goal) and one perturbation type (robot initial state) are studied; other perturbations (camera, lighting, layout) or other policies may leave more head-room.
- **Budget.** We test three random alternatives at up to three branch points per failed episode. With all 15 alternatives or more branch points the oracle bound would be higher, and we have not measured by how much. A single alternative succeeds from a failure state 2.7% of the time; if alternatives at a point succeeded independently, 15 of them would rescue up to about 34% of failure points. Outcomes from the same state are correlated, so this is optimistic, but it means our bound is specific to a budget of three alternatives and should not be read as a bound for exhaustive search.
- **One-chunk deviations.** This is the main limitation of the head-room analysis. Each branch deviates from System 1 for one chunk only and then returns control to it, so the rescue rate bounds single-chunk selection, not repeated selection over several consecutive decision points: recovery may require several coordinated chunks, and System 1 may undo a helpful intervention. Our selection experiments (Table 2) do select at every decision point, but with imperfect scorers, so they do not bound repeated selection either. The direct test is to branch with interventions of increasing horizon (alternatives in control for 1, 2, 4, … decision points, or until the end of the episode); this separates the recovery capability of the candidate generator from the constraints of the branching protocol, and we have released code for it.

- **Training data.** The scorers were trained on a few hundred rollouts; larger or same-state contrastive data might rank better. The head-room result limits what any such scorer can achieve with this proposer.

For practitioners, the branching protocol is a cheap check to run *before* investing in a verifier: if few failures are rescuable by the proposer's own candidates, effort is better spent on the policy itself, for example on recovery data in perturbed states [15], than on test-time selection.

8 Conclusion

A latent imagination search over a strong VLA's own action proposals did not improve robustness to perturbed robot initial states, under any of five scorers. Counterfactual branching shows that this is not merely a scorer problem: from the states where the policy fails, its proposals almost never lead to success, so even a perfect scorer choosing one chunk and then deferring to the policy would add only a few points; the ceiling for selection over several consecutive decisions remains to be measured. Test-time selection can only choose among the futures a policy can already reach; measuring how many of those futures succeed is a cheap first step before building a verifier.

References

[1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[4] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.

[5] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

[6] J. Cen, S. Huang, Y. Yuan, K. Li, H. Yuan, C. Yu, B. Hou, Y. Jiang, J. Guo, X. Li, H. Luo, F. Wang, D. Zhao, and H. Chen. RynnVLA-002: A unified vision-language-action and world model. *arXiv preprint arXiv:2511.17502*, 2025.

[7] J. Kwok, C. Agia, R. Sinha, M. Foutter, S. Li, I. Stoica, A. Mirhoseini, and M. Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. *arXiv preprint arXiv:2506.17811*, 2025.

[8] M. Nakamoto, O. Mees, A. Kumar, and S. Levine. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024.

[9] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, D. Zhao, and H. Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.

[10] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2018.

[11] N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.

[12] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

[13] J. E. Smith and R. L. Winkler. The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–321, 2006.

[14] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[15] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[16] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024.

[17] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[18] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

[19] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *arXiv preprint arXiv:2301.08243*, 2023.